

ELECTRONICS ENGINEERING

MICROPROCESSORS LABORATORY

2023-2024 FALL

EXPERIMENT-5

Group Members

Name	Fouad	Büşra	Ayşe Beril
Surname	Aladhami	Ülker	Çevik
Student ID	210102002808	200102002011	210102002020

1 INTRODUCTION **3**

2 PROBLEM 1 ..... **3**

2.1 FLOWCHART 4
2.2 SCHEMATIC 4
2.3 CODE 4

3 PROBLEM 2 ..... **10**

3.1 FLOWCHART 11
3.2 SCHEMATIC 12
3.3 CODE 12

4 CONCLUSION **21**

5 REFERENCES **21**

1 INTRODUCTION

[1] At problem using a ADC the analog signal from the connected pot read and this value is used to drive two led's connected to two PWM channels. The duty cycle of the PWM channels change according to the ADC value, so the brightness of the led's changed.

At Problem 2, the sound attenuator and acceleration sensor were used synchronously to count the knocks applied to the table on which the STM was placed. The knock count was displayed on the Seven Segment Display and reset with the button when necessary. The pins used for the sensors were selected by considering Figure 1.

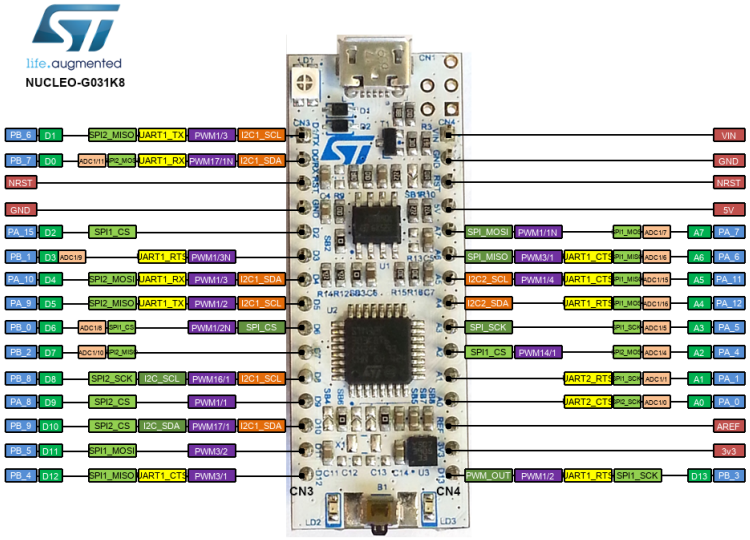
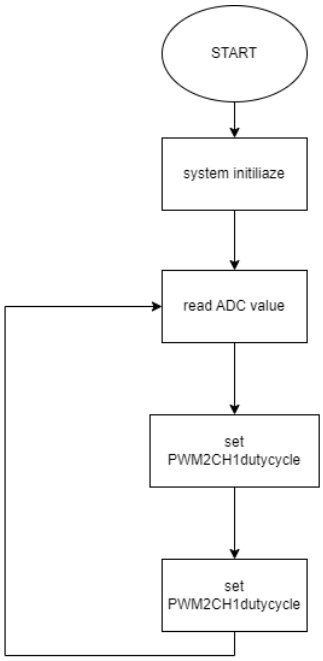


Figure 1. Stm32-G03K8 Pinout.

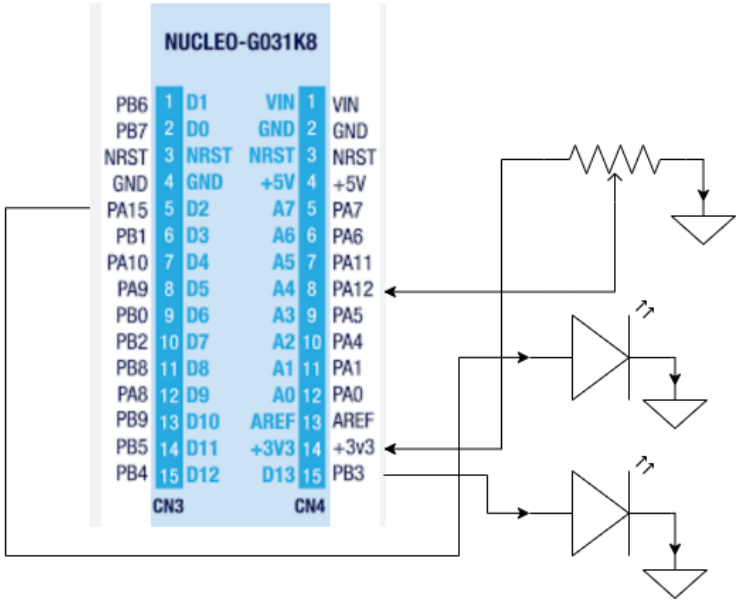
2 PROBLEM 1

The STM32G0 microcontroller reads analog signal with ADC from the pot connected to pin PA12, converts the ADC values to voltage, and set the duty cycles of the 2 PWM channels according to the ADC value. Each led is connected to PWM channel at pins PA15 and PB3. When the pot is turned to the left the ADC value will be 0 and the left led will bright with a full brightness (full duty cycle) when turning to the right the brightness of the left led decrease. At the middle the left led turn off, than when turning to the right the right led will turn on from low to high brightness according to the PWM signal duty cycle.

2.1 FLOWCHART



2.2 SCHEMATIC



2.3 CODE

```c
#include "stm32g0xx.h"
#include <math.h>
#include "main.h"
#include <stdio.h>
#include "stm32g031xx.h"
/*
implemnt a code that read adc from pa0, convert it to voltage
and detect a knock because this is a sound sensor
*/
// defines
#define VDDA 3.3
#define ADC_MAX_VALUE 4095
```

```c
#define FULL_DUTY_CYCLE 1000

// define prototypes
void RCC_init(void);
void GPIOA_init(void);
void GPIOB_init(void);
void ADC_init(void);
uint16_t poll_ADC(void);
void USART2_Init(void);
void printChar(uint8_t c);
int _print(int f, char *ptr, int len);
void print(char *s);
float convert_to_voltage(uint16_t adc_value);
/******PWM******/
void PWM2_CH1_init(void);
void PWM2_CH2_init(void);
void PWM2_StartPWM(void);
void PWM2_StopPWM(void);
void PWM2_CH1_SetDutyCycle(uint16_t dutyCycle);
void PWM2_CH2_SetDutyCycle(uint16_t dutyCycle);
void TIM2_IRQHandler(void);
uint16_t PWM2_CH1_calculate_Duty_cycle(uint16_t adcValue);
uint16_t PWM2_CH2_calculate_Duty_cycle(uint16_t adcValue);

// Systick functions prototypes
void delay(volatile uint32_t);
void Init_Systick(uint32_t tick);
void Systick_Handler(void);
void increase_tick(void);
uint64_t getTick(void);
void delay_ms(uint64_t msvalue);
void SysTick_Handler(void);

// Data types
typedef struct
{
	uint16_t adc_value;
	float voltage;
} ADC_typedef;

// variables
char txbuffer[6] = "/0";
uint16_t tackCounter = 0;

ADC_typedef pot;
// systick variables
uint64_t tick;

/**********PWM Variables**********/
uint16_t PWM2CH1dutyCycle;
uint16_t PWM2CH2dutyCycle;

int main(void)
{
	// Initialize the system
	RCC_init();
	GPIOA_init();
	GPIOB_init();
	ADC_init();
	PWM2_CH1_init();
	PWM2_CH2_init();
	USART2_Init();
	Init_Systick(16000);
	PWM2_CH2_SetDutyCycle(0);
	PWM2_StartPWM();
	// Infinite loop
	while (1)
	{
```

```
                // Read the ADC converted value
                pot.adc_value = poll_ADC();
                // Convert adc value to voltage
                pot.voltage = convert_to_voltage(pot.adc_value);
                PWM2CH1dutyCycle = PWM2_CH1_calculate_Duty_cycle(pot.adc_value);
                PWM2CH2dutyCycle = PWM2_CH2_calculate_Duty_cycle(pot.adc_value);
                sprintf(txbuffer, "%d", pot.adc_value);
                print(txbuffer);
                print("\n");
        }
        return 0;
}
void RCC_init(void)
{
        RCC->IOPENR |= RCC_IOPENR_GPIOAEN;
        // Enable ADC clock
        RCC->APBENR2 |= RCC_APBENR2_ADCEN;

        /************* Uart *************/
        // enable GPIOA and USART2 module clocks from RCC
        RCC->IOPENR |= RCC_IOPENR_GPIOAEN;
        RCC->APBENR1 |= RCC_APBENR1_USART2EN;

        /***************PWM*****************/
        // Enable GPIOB clock
        RCC->IOPENR |= (1U << 1);
        // Enable TIM2 clock
        RCC->APBENR1 |= RCC_APBENR1_TIM2EN;
}

void GPIOA_init(void)
{
        // Pa12 pin as input(ADC) PA12
        GPIOA->MODER &= ~GPIO_MODER_MODE12;
        // Enable the adc mode
        GPIOA->MODER |= (GPIO_MODER_MODE12_1 | GPIO_MODER_MODE12_0);
        // Pa12 pin as push pull
        GPIOA->OTYPER &= ~GPIO_OTYPER_OT12;
        // Pa12 pin as high speed
        GPIOA->OSPEEDR &= ~GPIO_OSPEEDR_OSPEED12;
        // Pa12 pin as no pull up, no pull down
        GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD12;

        /**********UART************/
        // enable and setup pins PA2 and PA3 as alternate function for USART2
module
        GPIOA->MODER &= ~(GPIO_MODER_MODE2 | GPIO_MODER_MODE3);
        GPIOA->MODER |= (GPIO_MODER_MODE2_1 | GPIO_MODER_MODE3_1);
        GPIOA->AFR[0] &= ~(GPIO_AFRL_AFSEL2 | GPIO_AFRL_AFSEL3);
        GPIOA->AFR[0] |= (GPIO_AFRL_AFSEL2_0 | GPIO_AFRL_AFSEL3_0);

        /***********PWM************/
        // Set alternate function to 2
        // 15 comes from PA15
        GPIOA->AFR[1] |= (GPIO_AFRH_AFSEL15_1);
        // Set pwm for PA15
        GPIOA->MODER &= ~(GPIO_MODER_MODE15);

        // Select AF from Moder
        GPIOA->MODER |= (GPIO_MODER_MODE15_1);
}
void GPIOB_init(void)
{
        /***********PWM************/
        // Set alternate function to 2
        // 3 comes from PB3
        GPIOB->AFR[0] |= (GPIO_AFRL_AFSEL3_1);
        // Select AF from Moder
```

```
	GPIOB->MODER &= ~(GPIO_MODER_MODE3);
	GPIOB->MODER |= (GPIO_MODER_MODE3_1);
}
void ADC_init(void)
{
	// Set ADC clock source to PCLK/2
	ADC1->CFGR2 &= ~ADC_CFGR2_CKMODE;
	// Enable the ADC voltage regulator
	ADC1->CR |= ADC_CR_ADVREGEN;
	// Wait for the ADC voltage regulator startup time
	for (int i = 0; i < 1000; i++)
		;
	// Calibrate the ADC
	ADC1->CR |= ADC_CR_ADCAL;
	// Wait for the end of ADC calibration
	while (ADC1->CR & ADC_CR_ADCAL)
		;
	// Enable the ADC
	ADC1->ISR |= ADC_ISR_ADRDY;
	// Enable the ADC
	ADC1->CR |= ADC_CR_ADEN;
	// Wait until ADC ready
	while (!(ADC1->ISR & ADC_ISR_ADRDY))
		;
	// Select the ADC input channel
	ADC1->CHSELR |= ADC_CHSELR_CHSEL16;
}
void USART2_Init(void)
{
	// enable receive and transmit from USART2 module
	USART2->CR1 |= (USART_CR1_RE | USART_CR1_TE);
	// set baud rate to 9600 bps assuming clock is running at 16Mhz
	// Baud Rate = APBxCLK / USARTx->BRR
	USART2->BRR = 0x683; // 1667
	// enable uart module
	USART2->CR1 |= USART_CR1_UE;
}

uint16_t poll_ADC(void)
{
	// Start the ADC conversion
	ADC1->CR |= ADC_CR_ADSTART;
	// Wait for the end of the conversion
	while (!(ADC1->ISR & ADC_ISR_EOC))
		;
	// Read the ADC converted value
	return ADC1->DR;
}

// Convert adc value to voltage
float convert_to_voltage(uint16_t adc_value)
{
	return (float)adc_value * VDDA / ADC_MAX_VALUE;
}

/*********PWM***********/

void TIM2_IRQHandler(void)
{

	// update duty
	PWM2_CH1_SetDutyCycle(PWM2CH1dutyCycle);
	PWM2_CH2_SetDutyCycle(PWM2CH2dutyCycle);

	// Clear update status register
	TIM2->SR &= ~(1U << 0);
}
```

```
void PWM2_CH1_init(void)
{
	// zero out the control register just in case
	TIM2->CR1 = 0;

	// Select PWM Mode 1
	TIM2->CCMR1 |= TIM_CCMR1_OC1M_1 | TIM_CCMR1_OC1M_2;
	// Preload Enable
	TIM2->CCMR1 |= TIM_CCMR1_OC1PE;

	// Capture compare ch1 enable
	TIM2->CCER |= TIM_CCER_CC1E;

	// zero out counter
	TIM2->CNT = 0;
	// 1 ms interrupt
	// CLOCK/((PSC+1)* ARR)
	TIM2->PSC = 15;
	TIM2->ARR = 1000;

	// zero out duty
	TIM2->CCR1 = 0;

	// Update interrupt enable
	TIM2->DIER |= (1 << 0);

	// TIM1 Enable
	// TIM2->CR1 |= TIM_CR1_CEN;

	NVIC_SetPriority(TIM2_IRQn, 1);
	NVIC_EnableIRQ(TIM2_IRQn);
}

/*
 * Setup PWM output for
 * TIM2 CH2 on PB3
 */
void PWM2_CH2_init(void)
{
	// zero out the control register just in case
	TIM2->CR1 = 0;

	// Select PWM Mode 1
	TIM2->CCMR1 |= (6U << 12);
	// Preload Enable
	TIM2->CCMR1 |= TIM_CCMR1_OC2PE;

	// Capture compare ch2 enable
	TIM2->CCER |= TIM_CCER_CC2E;

	// zero out counter
	TIM2->CNT = 0;
	// 1 ms interrupt
	// CLOCK/((PSC+1)* ARR)
	TIM2->PSC = 15;
	TIM2->ARR = 1000;

	// zero out duty
	TIM2->CCR2 = 0;

	// Update interrupt enable
	TIM2->DIER |= (1 << 0);

	// TIM1 Enable
	// TIM2->CR1 |= TIM_CR1_CEN;

	NVIC_SetPriority(TIM2_IRQn, 1);
	NVIC_EnableIRQ(TIM2_IRQn);
```

```
}
void PWM2_StartPWM(void)
{
	// Enable the timer
	TIM2->CR1 |= TIM_CR1_CEN;
}

void PWM2_StopPWM(void)
{
	// Disable the timer
	TIM2->CR1 &= ~TIM_CR1_CEN;
}
void PWM2_CH1_SetDutyCycle(uint16_t dutyCycle)
{
	// Ensure duty cycle is within bounds
	// Set the duty cycle by updating the CCR1 register
	TIM2->CCR1 = dutyCycle;
}
void PWM2_CH2_SetDutyCycle(uint16_t dutyCycle)
{
	// Ensure duty cycle is within bounds
	// Set the duty cycle by updating the CCR2 register
	TIM2->CCR2 = dutyCycle;
}

uint16_t PWM2_CH1_calculate_Duty_cycle(uint16_t adcValue)
{
	uint16_t dutyCycle = 0;

	if (adcValue < (ADC_MAX_VALUE / 2))
	{
		dutyCycle = 0;
	}
	else
	{
		dutyCycle = (uint16_t)round((((float)(adcValue * 2 -
ADC_MAX_VALUE)) / ADC_MAX_VALUE) * FULL_DUTY_CYCLE);
	}
	return dutyCycle;
}
// calculate duty cycle for the first led near the ground
uint16_t PWM2_CH2_calculate_Duty_cycle(uint16_t adcValue)
{
	uint16_t dutyCycle = 0;

	if (adcValue > (ADC_MAX_VALUE / 2))
	{
		dutyCycle = 0;
	}
	else
	{
		dutyCycle = FULL_DUTY_CYCLE - (uint16_t)round((((float)adcValue *
2) / ADC_MAX_VALUE) * FULL_DUTY_CYCLE);
	}
	return dutyCycle;
}

/*********Uart functions**********/

void printChar(uint8_t c)
{
	// write c to the transmit data register
	USART2->TDR = c;
	// wait until transmit is complete
	while (!(USART2->ISR & USART_ISR_TC))
		;
}
int _print(int f, char *ptr, int len)
```

```
{
	(void)f; // don't do anthing with f
	// call printChar within a for loop len times
	for (int i = 0; i < len; i++)
	{
		printChar(ptr[i]);
	}
	return len; // return length
}
void print(char *s)
{
	// count number of characters in s string until a null byte comes `\0`
	int length = 0;
	while (s[length] != '\0')
	{
		length++;
	}
	_print(0, s, length);
}

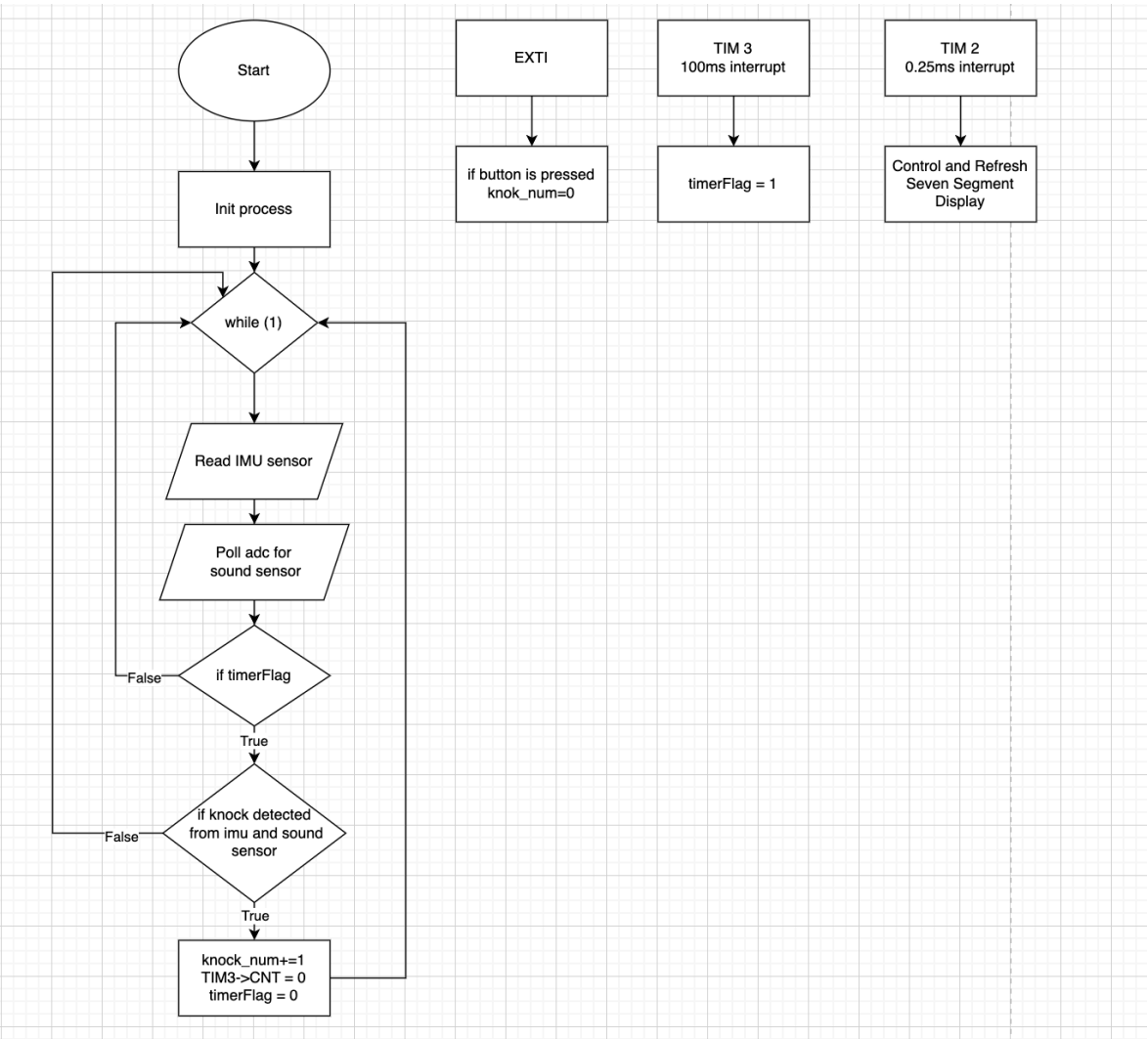
void increase_tick(void)
{
	tick++;
}
void Init_Systick(uint32_t tick)
{
	SysTick->LOAD = tick; // Count down from 999 to 0
	SysTick->VAL = 0;     // Clear current value
	SysTick->CTRL = 0x7;  // Enable Systick, exception,and use processor
clock
}
uint64_t getTick(void)
{
	return tick;
}
void delay_ms(uint64_t msvalue)
{
	uint64_t startTick = getTick();
	while ((getTick() - startTick) < msvalue)
	{
	}
}
/**
 * @brief This function handles System tick timer.
 */
void SysTick_Handler(void)
{
	increase_tick();
}
```

3 PROBLEM 2

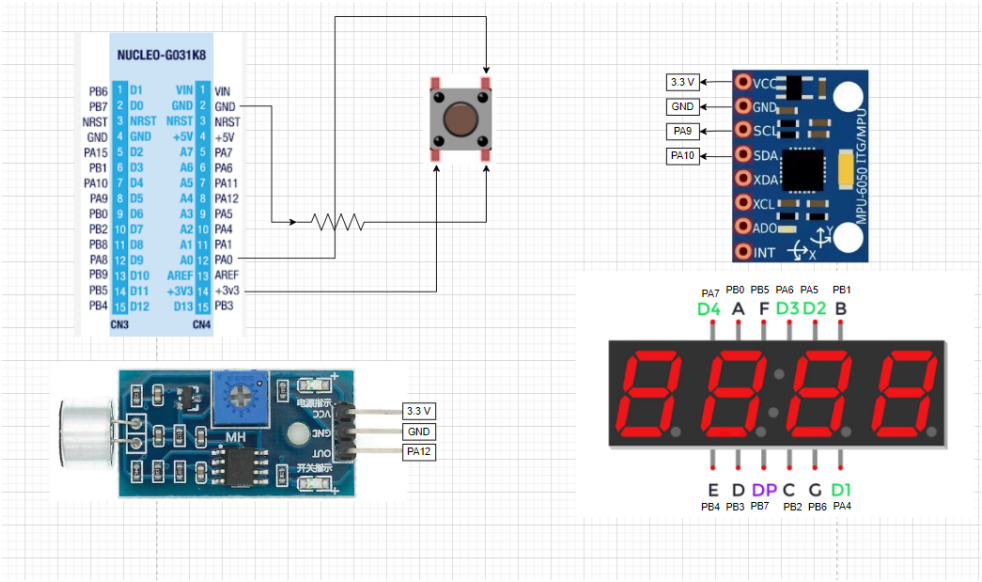
In this problem, the sound sensor and the IMU sensor is combined to detect knock and write the number of the knocks to the seven-segment display. The IMU sensor data, acceleration and gyro for each axis is read using I2C protocol. The gyro is measuring the angle velocity from 0 to 250 degree per second. Always the sensor is read, and smooth data is obtained by using exponential smoothing, by a factor of 0.001. When the instant value of the gyro in one of any axes is far then the smooth gyro data 20 degree/second, it's mean that a vibration detected and it's that the IMU say that is a possible knock. Then the sound data is read from sound sensor if a high sound detected that is a possible knock. If at the same time the IMU sensor and the sound sensor detect a possible knock

that’s mean that’s really knock and the knock counter increase. Then the timer flag is set to zero and timer 3 counter set to 0. After 100ms the timer flag will set to 1 inside the interrupt. The knock check will only be done if the flag value is 1. This method avoid blocking delay and the debouncing of knock checking.

3.1 FLOWCHART



3.2 SCHEMATIC



3.3 CODE

```
#include "stm32g0xx.h"
#include <math.h>
#include "main.h"
#include <stdio.h>
#include <stdlib.h>
#include "stm32g031xx.h"
/*
implemnt a code that read adc from pa0, convert it to voltage
and detect a knock because this is a sound sensor
*/

/*******Sound Sensor*******/
// defines
#define VDDA 3.3
#define ADC_MAX_VALUE 4095

// variables

typedef struct
{
	uint16_t adc_value;
	float voltage;
} sound_sensor;

sound_sensor soundSensor;

/*******Seven segment*******/

// seven segment display defines
#define SEGMENT_A 1U << (1U - 1U)
#define SEGMENT_B 1U << (2U - 1U)
#define SEGMENT_C 1U << (3U - 1U)
#define SEGMENT_D 1U << (4U - 1U)
#define SEGMENT_E 1U << (5U - 1U)
#define SEGMENT_F 1U << (6U - 1U)
#define SEGMENT_G 1U << (7U - 1U)
#define SEGMENT_DP 1U << (8U - 1U)
#define SEGMENT_MASK ~(SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E |
SEGMENT_F | SEGMENT_G | SEGMENT_DP)

#define DIGIT_1 1 << (4)
#define DIGIT_2 1 << (5)
```

```
#define DIGIT_3 1 << (6)
#define DIGIT_4 1 << (7)

#define DIGIT_MASK (DIGIT_1 | DIGIT_2 | DIGIT_3 | DIGIT_4)

// seven segment display numbers
uint8_t numbers[10] = {
       SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E | SEGMENT_F,
                 // 0
       SEGMENT_B | SEGMENT_C,
                                                          // 1
       SEGMENT_A | SEGMENT_B | SEGMENT_D | SEGMENT_E | SEGMENT_G,
                                 // 2
       SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_G,
                                 // 3
       SEGMENT_B | SEGMENT_C | SEGMENT_F | SEGMENT_G,
                                       // 4
       SEGMENT_A | SEGMENT_C | SEGMENT_D | SEGMENT_F | SEGMENT_G,
                                 // 5
       SEGMENT_A | SEGMENT_C | SEGMENT_D | SEGMENT_E | SEGMENT_F | SEGMENT_G,
                 // 6
       SEGMENT_A | SEGMENT_B | SEGMENT_C,
                                             // 7
       SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_E | SEGMENT_F |
SEGMENT_G, // 8
       SEGMENT_A | SEGMENT_B | SEGMENT_C | SEGMENT_D | SEGMENT_F | SEGMENT_G,
                 // 9
};
// defines
#define TIMERPSC 160
#define TIMERPERIYOD 25
// variables
uint64_t tick;
uint16_t numberCounter;
uint32_t counter;
uint8_t buttonIsPressed;

// define prototypes
void RCC_init(void);
void GPIOA_init(void);
void GPIOB_Init(void);

/*****ADC functions *******/
void ADC_init(void);
uint16_t poll_ADC(void);
static inline float convert_to_voltage(uint16_t adc_value);

/****** seven segment******/
void TIM2_Clock_Init(void);
void TIM2_Interrupt_Config(void);
void TIM2_IRQHandler(void);
void Start_TIM2(void);
void Stop_TIM2(void);

void EXTI_Init(void);
void EXTI0_1_IRQHandler(void);
void sevenSegmentShowDigit(uint8_t number, uint8_t digit);

/*******IMU*******/

#define MPU6050_ADDRESS 0x68

#define MPU6050_WHO_AM_I 0x75
#define MPU6050_POWER_MGMT_1 0x6B

// ACCELEROMETER REGISTER ADDRESS
```

```c
#define MPU6050_ACCEL_XOUT_H 0x3B
#define MPU6050_ACCEL_XOUT_L 0x3C

#define MPU6050_ACCEL_YOUT_H 0x3D
#define MPU6050_ACCEL_YOUT_L 0x3E

#define MPU6050_ACCEL_ZOUT_H 0x3F
#define MPU6050_ACCEL_ZOUT_L 0x40

// GYRO REGISTER ADDRESS
#define MPU6050_GYRO_XOUT_H 0x43
#define MPU6050_GYRO_XOUT_L 0x44

#define MPU6050_GYRO_YOUT_H 0x45
#define MPU6050_GYRO_YOUT_L 0x46

#define MPU6050_GYRO_ZOUT_H 0x47
#define MPU6050_GYRO_ZOUT_L 0x48

#define SMOOTHING_FACTOR 0.001
#define KNOCK_THRESHOLD 20
#define TIMER3PSC 16000
#define TIMER3PERIYOD 100

typedef struct
{
	float accelx;
	float accely;
	float accelz;
	float gyrox;
	float gyroy;
	float gyroz;

	float accelx_smooth;
	float accely_smooth;
	float accelz_smooth;
	float gyrox_smooth;
	float gyroy_smooth;
	float gyroz_smooth;
} MPU6050;

void I2C_Config(void);
uint8_t I2C_Read(uint8_t, uint8_t);
void I2C_Write(uint8_t devAddr, uint8_t regAddr, uint8_t data);

uint8_t detect_knock(MPU6050 sensorData);

void TIM3_Clock_Init(void);
void TIM3_Interrupt_Config(void);
void TIM3_IRQHandler(void);
void Start_TIM3(void);
void Stop_TIM3(void);
void IMU_sensor_read(void);

int16_t data;
int16_t accelx;
int16_t gyrox;
int16_t accely;
int16_t gyroy;
int16_t accelz;
int16_t gyroz;

uint32_t knock_num;
uint8_t timerFlag;

MPU6050 sensorData;
```

```c
int main(void)
{
	// Initialize the system
	RCC_init();
	GPIOA_init();
	GPIOB_Init();
	ADC_init();
	I2C_Config();

	// Show 0 at initial condition
	sevenSegmentShowDigit(0, 3);
	TIM2_Clock_Init();
	TIM2_Interrupt_Config();
	TIM3_Clock_Init();
	TIM3_Interrupt_Config();
	Start_TIM3();
	EXTI_Init();

	Start_TIM2();

	data = I2C_Read(MPU6050_ADDRESS, MPU6050_WHO_AM_I);
	data = I2C_Read(MPU6050_ADDRESS, MPU6050_POWER_MGMT_1);
	I2C_Write(MPU6050_ADDRESS, MPU6050_POWER_MGMT_1, 0x00);

	data = I2C_Read(MPU6050_ADDRESS, MPU6050_POWER_MGMT_1);

	// Infinite loop
	while (1)
	{
		IMU_sensor_read();
		// Read the ADC converted value
		soundSensor.adc_value = poll_ADC();
		// Convert adc value to voltage
		soundSensor.voltage = convert_to_voltage(soundSensor.adc_value);

		if (timerFlag)
		{
			if (detect_knock(sensorData) && soundSensor.adc_value > 3000)
			{
				knock_num += 1;
				TIM3->CNT = 0;
				timerFlag = 0;
			}
		}
	}
	return 0;
}
/************ IRQ HANDLERS ************/
void TIM2_IRQHandler(void)
{
	if (TIM2->SR & TIM_SR_UIF)
	{
		if (numberCounter < 4)
		{
			numberCounter++;
			counter = knock_num;
		}
		else
		{
			numberCounter = 1;
		}

		uint8_t digit1 = counter / 1000;
		uint8_t digit2 = (counter % 1000) / 100;
		uint8_t digit3 = (counter % 100) / 10;
```

```
                uint8_t digit4 = counter % 10;
                if (numberCounter % 4 == 0)
                {
                        sevenSegmentShowDigit(digit4, 0);
                }
                else if (numberCounter % 3 == 0)
                {
                        sevenSegmentShowDigit(digit3, 1);
                }
                else if (numberCounter % 2 == 0)
                {
                        sevenSegmentShowDigit(digit2, 2);
                }
                else if (numberCounter % 1 == 0)
                {
                        sevenSegmentShowDigit(digit1, 3);
                }


                // toggle_led();

                // Clear the interrupt flag
                TIM2->SR &= ~TIM_SR_UIF;
        }
}
void EXTI0_1_IRQHandler(void)
{
        // Your code to handle the EXTI interrupt for PA0 goes here
        // This is where you can perform specific actions when the rising edge occurs
        if (EXTI->RPR1 & EXTI_FPR1_FPIF0) // Check if the interrupt is from line 0
        {
                buttonIsPressed = 1;
                knock_num = 0;

                // Clear EXTI0 pending flag
                EXTI->RPR1 |= EXTI_FPR1_FPIF0;
        }
}
/*********** RCC and GPIOS functions ************/

void RCC_init(void)
{
        /* Enable GPIOA ,B and C clock */
        RCC->IOPENR |= RCC_IOPENR_GPIOAEN | RCC_IOPENR_GPIOBEN | RCC_IOPENR_GPIOCEN;
        // Enable ADC clock
        RCC->APBENR2 |= RCC_APBENR2_ADCEN;

        // Enable clock for SYSCFG
        RCC->APBENR2 |= RCC_APBENR2_SYSCFGEN;

        // Enables GPIOA peripheral
        RCC->AHBENR |= 1;
}

void GPIOA_init(void)
{
        // Pa12 pin as input
        GPIOA->MODER &= ~GPIO_MODER_MODE12;
        // Enable the adc mode
        GPIOA->MODER |= (GPIO_MODER_MODE12_1 | GPIO_MODER_MODE12_0);
        // Pa12 pin as push pull
        GPIOA->OTYPER &= ~GPIO_OTYPER_OT12;
        // Pa12 pin as high speed
        GPIOA->OSPEEDR &= ~GPIO_OSPEEDR_OSPEED12;
        // Pa12 pin as no pull up, no pull down
        GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD12;
```

```
        /***** Sevent segment configs *****/
        // Set GPIOA Pin 0 as input
        GPIOA->MODER &= ~GPIO_MODER_MODE0; // Clear mode bits for pin 0
        // clear GPIOA Pin 4,5,6,7
        GPIOA->MODER &= ~GPIO_MODER_MODE4; // Clear mode bits for pin 4
        GPIOA->MODER &= ~GPIO_MODER_MODE5; // Clear mode bits for pin 5
        GPIOA->MODER &= ~GPIO_MODER_MODE6; // Clear mode bits for pin 6
        GPIOA->MODER &= ~GPIO_MODER_MODE7; // Clear mode bits for pin 7
        //  Set GPIOA Pin 4,5,6,7 as output
        GPIOA->MODER |= (1U << 2 * 4); // Set mode bits for pin 4
        GPIOA->MODER |= (1U << 2 * 5); // Set mode bits for pin 5
        GPIOA->MODER |= (1U << 2 * 6); // Set mode bits for pin 6
        GPIOA->MODER |= (1U << 2 * 7); // Set mode bits for pin 7
        // Enable pull-down resistor for GPIOA Pin 0
        GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0;  // Clear pull-up/pull-down bits for pin 0
        GPIOA->PUPDR |= GPIO_PUPDR_PUPD0_1; // Set bit for pull-down

        /*********MPU*********/
        // Select AF from Moder for PA9,PA10 (10)
        GPIOA->MODER &= ~(3U << 2 * 9);
        GPIOA->MODER |= (2U << 2 * 9);
        GPIOA->OTYPER |= (1U << 9);
        GPIOA->OSPEEDR |= (3U << 18); // high speed
        GPIOA->PUPDR |= (1U << 18);
        GPIOA->AFR[1] |= (6U << 4); // for PA9,PA10 AF6(I2C)

        GPIOA->MODER &= ~(3U << 2 * 10);
        GPIOA->MODER |= (2U << 2 * 10);
        GPIOA->OTYPER |= (1U << 10);
        GPIOA->OSPEEDR |= (3U << 20); // high speed
        GPIOA->PUPDR |= (1U << 20);
        GPIOA->AFR[1] |= (6U << 8); // for PA9,PA10 AF6(I2C)
}
void GPIOB_Init(void)
{
        // clear GPIOB Pin 0,1,2,3,4,5,6,7,8
        GPIOB->MODER &= SEGMENT_MASK & SEGMENT_MASK << 8; // Clear mode bits for pin 0
        // Set GPIOB Pin 0,1,2,3,4,5,6,7,8 as output
        GPIOB->MODER |= (1U << 2 * 0); // Set mode bits for pin 0
        GPIOB->MODER |= (1U << 2 * 1); // Set mode bits for pin 1
        GPIOB->MODER |= (1U << 2 * 2); // Set mode bits for pin 2
        GPIOB->MODER |= (1U << 2 * 3); // Set mode bits for pin 3
        GPIOB->MODER |= (1U << 2 * 4); // Set mode bits for pin 4
        GPIOB->MODER |= (1U << 2 * 5); // Set mode bits for pin 5
        GPIOB->MODER |= (1U << 2 * 6); // Set mode bits for pin 6
        GPIOB->MODER |= (1U << 2 * 7); // Set mode bits for pin 7
        GPIOB->MODER |= (1U << 2 * 8); // Set mode bits for pin 8
}

/**************ADC functions**************/

void ADC_init(void)
{
        // Set ADC clock source to PCLK/2
        ADC1->CFGR2 &= ~ADC_CFGR2_CKMODE;
        // Enable the ADC voltage regulator
        ADC1->CR |= ADC_CR_ADVREGEN;
        // Wait for the ADC voltage regulator startup time
        for (int i = 0; i < 1000; i++)
                ;
        // Calibrate the ADC
        ADC1->CR |= ADC_CR_ADCAL;
        // Wait for the end of ADC calibration
        while (ADC1->CR & ADC_CR_ADCAL)
                ;
```

```
        // Enable the ADC
        ADC1->ISR |= ADC_ISR_ADRDY;
        // Enable the ADC
        ADC1->CR |= ADC_CR_ADEN;
        // Wait until ADC ready
        while (!(ADC1->ISR & ADC_ISR_ADRDY))
                ;
        // Select the ADC input channel
        ADC1->CHSELR |= ADC_CHSELR_CHSEL16;
}
uint16_t poll_ADC(void)
{
        // Start the ADC conversion
        ADC1->CR |= ADC_CR_ADSTART;
        // Wait for the end of the conversion
        while (!(ADC1->ISR & ADC_ISR_EOC))
                ;
        // Read the ADC converted value
        return ADC1->DR;
}
// Convert adc value to voltage
static inline float convert_to_voltage(uint16_t adc_value)
{
        return (float)adc_value * VDDA / ADC_MAX_VALUE;
}

/********** Seven segment functions***********/
void EXTI_Init(void)
{

        // Connect EXTI1 to GPIOA Pin 0
        EXTI->EXTICR[1] |= EXTI_EXTICR1_EXTI0_0;

        // Configure EXTI0 to rising edge trigger
        EXTI->RTSR1 |= EXTI_RTSR1_RT0;
        EXTI->FTSR1 &= ~EXTI_FTSR1_FT0;

        // Enable EXTI0 interrupt
        EXTI->IMR1 |= EXTI_IMR1_IM0;

        // Set the EXTI0 interrupt priority (adjust as needed)
        NVIC_SetPriority(EXTI0_1_IRQn, 1);
        NVIC_EnableIRQ(EXTI0_1_IRQn);
}

void TIM2_Clock_Init(void)
{
        // Enable TIM2 clock
        RCC->APBENR1 |= RCC_APBENR1_TIM2EN;

        // Set TIM2 prescaler and period
        TIM2->PSC = TIMERPSC - 1;
        TIM2->CNT = 0;
        TIM2->ARR = (TIMERPERIYOD)-1;
}
void TIM2_Interrupt_Config(void)
{
        NVIC_SetPriority(TIM2_IRQn, 15);
        NVIC_EnableIRQ(TIM2_IRQn);
        // Update interrupt enable
        TIM2->DIER |= TIM_DIER_UIE;
}

void Start_TIM2(void)
{
        // Start TIM2
```

```
        TIM2->CR1 |= TIM_CR1_CEN;
}
void Stop_TIM2(void)
{
        // Stop TIM2
        TIM2->CR1 &= ~TIM_CR1_CEN;
}

void sevenSegmentShowDigit(uint8_t number, uint8_t digit)
{
        // select digit4 using GPIOA with mask
        GPIOA->ODR |= DIGIT_MASK;

        // set GPIOB to output for digit4 using numbers with 8 bit mask
        GPIOB->ODR = (GPIOB->ODR & SEGMENT_MASK) | numbers[number];
        GPIOA->ODR &= ~(1 << (digit + 4));
}

/********** IMU functions************/

void I2C_Config(void)
{
        // Enable the I2C Clock and GPIO Clock
        RCC->APBENR1 |= (1U << 21); // enable IC2 clock
        // Enables GPIOA peripheral
        RCC->AHBENR |= 1;
        I2C1->CR1 |= 0;

        I2C1->TIMINGR |= (3 << 28);    // PRESC
        I2C1->TIMINGR |= (0x13 << 0); // SCLL
        I2C1->TIMINGR |= (0xF << 8);  // SCLH
        I2C1->TIMINGR |= (0x2 << 16); // SDADEL
        I2C1->TIMINGR |= (0x4 << 20); // SCLDEL

        I2C1->CR1 = (1U << 0); // PE

        NVIC_SetPriority(I2C1_IRQn, 1);
        NVIC_EnableIRQ(I2C1_IRQn);
}

uint8_t I2C_Read(uint8_t devAddr, uint8_t regAddr)
{

        // WRITE OPERTAION(Send address and register to read)
        I2C1->CR2 = 0;
        I2C1->CR2 |= ((uint32_t)devAddr << 1); // slave address
        I2C1->CR2 |= (1U << 16);                        // Number of bytes
        I2C1->CR2 |= (1U << 13);                        // Generate Start
        while (!(I2C1->ISR & (1 << 1)))
                ; // TXIS

        I2C1->TXDR = (uint32_t)regAddr;
        while (!(I2C1->ISR & (1 << 6)))
                ; // TC

        // READ OPERATION(read data)
        I2C1->CR2 = 0;
        I2C1->CR2 |= ((uint32_t)devAddr << 1); // slave address
        I2C1->CR2 |= (1U << 10);                        // READ mode
        I2C1->CR2 |= (1U << 16);                        // Number of bytes
        I2C1->CR2 |= (1U << 15);                        // NACK
        I2C1->CR2 |= (1U << 25);                        // AUTOEND

        I2C1->CR2 |= (1U << 13); // Generate Start
        while (!(I2C1->ISR & (1 << 2)))
                ; // wait until RXNE=1
```

```
	uint8_t data = (uint8_t)I2C1->RXDR;
	return data;
}

void I2C_Write(uint8_t devAddr, uint8_t regAddr, uint8_t data)
{

	// WRITE OPERTAION(Send address and register to read)
	I2C1->CR2 = 0;
	I2C1->CR2 |= ((uint32_t)devAddr << 1); // slave address
	I2C1->CR2 |= (2U << 16);                    // Number of bytes
	I2C1->CR2 |= (1U << 25);                    // AUTOEND
	I2C1->CR2 |= (1U << 13);                    // Generate Start

	while (!(I2C1->ISR & (1 << 1)))
		; // TXIS
	I2C1->TXDR = (uint32_t)regAddr;

	while (!(I2C1->ISR & (1 << 1)))
		; // TXIS
	I2C1->TXDR = (uint32_t)data;
}

//To control IMU sensor data
uint8_t detect_knock(MPU6050 sensorData)
{
	if (abs(sensorData.gyrox - sensorData.gyrox_smooth) > KNOCK_THRESHOLD ||
abs(sensorData.gyrox - sensorData.gyroy_smooth) > KNOCK_THRESHOLD ||
abs(sensorData.gyroz - sensorData.gyroz_smooth) > KNOCK_THRESHOLD)
	{
		return 1;
	}
	else
	{
		return 0;
	}
}
void TIM3_Clock_Init(void)
{
	// Enable TIM3 clock
	RCC->APBENR1 |= RCC_APBENR1_TIM3EN;

	// Set TIM3 prescaler and period
	TIM3->PSC = TIMER3PSC - 1;
	TIM3->CNT = 0;
	TIM3->ARR = TIMER3PERIYOD - 1;
}
void TIM3_Interrupt_Config(void)
{
	NVIC_SetPriority(TIM3_IRQn, 15);
	NVIC_EnableIRQ(TIM3_IRQn);
	// Update interrupt enable
	TIM3->DIER |= TIM_DIER_UIE;
}

void Start_TIM3(void)
{
	// Start TIM3
	TIM3->CR1 |= TIM_CR1_CEN;
}
void Stop_TIM3(void)
{
	// Stop TIM3
	TIM3->CR1 &= ~TIM_CR1_CEN;
}
```

```
void TIM3_IRQHandler(void)
{
	if (TIM3->SR & TIM_SR_UIF)
	{
		// Code here
		timerFlag = 1;
		// Clear the interrupt flag
		TIM3->SR &= ~TIM_SR_UIF;
	}
}
// To read IMU sensor data and store
void IMU_sensor_read(void)
{

	accelx = I2C_Read(MPU6050_ADDRESS, MPU6050_ACCEL_XOUT_L);
	accelx = accelx | (I2C_Read(MPU6050_ADDRESS, MPU6050_ACCEL_XOUT_H) << 8);
	sensorData.accelx = (float)(accelx) / 16384;
	sensorData.accelx_smooth += SMOOTHING_FACTOR * (sensorData.accelx -
sensorData.accelx_smooth);

	accely = I2C_Read(MPU6050_ADDRESS, MPU6050_ACCEL_YOUT_L);
	accely = accely | (I2C_Read(MPU6050_ADDRESS, MPU6050_ACCEL_YOUT_H) << 8);
	sensorData.accely = (float)(accely) / 16384;
	sensorData.accely_smooth += SMOOTHING_FACTOR * (sensorData.accely -
sensorData.accely_smooth);

	accelz = I2C_Read(MPU6050_ADDRESS, MPU6050_ACCEL_ZOUT_L);
	accelz = accelz | (I2C_Read(MPU6050_ADDRESS, MPU6050_ACCEL_ZOUT_H) << 8);
	sensorData.accelz = (float)(accelz) / 16384;
	sensorData.accelz_smooth += SMOOTHING_FACTOR * (sensorData.accelz -
sensorData.accelz_smooth);

	gyrox = I2C_Read(MPU6050_ADDRESS, MPU6050_GYRO_XOUT_L);
	gyrox = gyrox | (I2C_Read(MPU6050_ADDRESS, MPU6050_GYRO_XOUT_H) << 8);
	sensorData.gyrox = (float)(gyrox) / 131;
	sensorData.gyrox_smooth += SMOOTHING_FACTOR * (sensorData.gyrox -
sensorData.gyrox_smooth);

	gyroy = I2C_Read(MPU6050_ADDRESS, MPU6050_GYRO_YOUT_L);
	gyroy = gyroy | (I2C_Read(MPU6050_ADDRESS, MPU6050_GYRO_YOUT_H) << 8);
	sensorData.gyroy = (float)(gyroy) / 131;
	sensorData.gyroy_smooth += SMOOTHING_FACTOR * (sensorData.gyroy -
sensorData.gyroy_smooth);

	gyroz = I2C_Read(MPU6050_ADDRESS, MPU6050_GYRO_ZOUT_L);
	gyroz = gyroz | (I2C_Read(MPU6050_ADDRESS, MPU6050_GYRO_ZOUT_H) << 8);
	sensorData.gyroz = (float)(gyroz) / 131;
	sensorData.gyroz_smooth += SMOOTHING_FACTOR * (sensorData.gyroz -
sensorData.gyroz_smooth);
}
```

4 CONCLUSION

As a result, in Problem 1, the duty cycle value of PWM was set with ADC, while in Problem 2, the sound sensor was operated with the help of ADC, IMU I2C and data was obtained. Then the tasks were fulfilled according to the data received by using interrupts such as TIM, Systick, EXTI used in the microprocessor laboratory. By merging IMU and the sound sensor the knock is detected with higher accuracy.

5 REFERENCES

[1]rm0444-stm32g0x1-advanced-armbased-32bit-mcus-stmicroelectronics